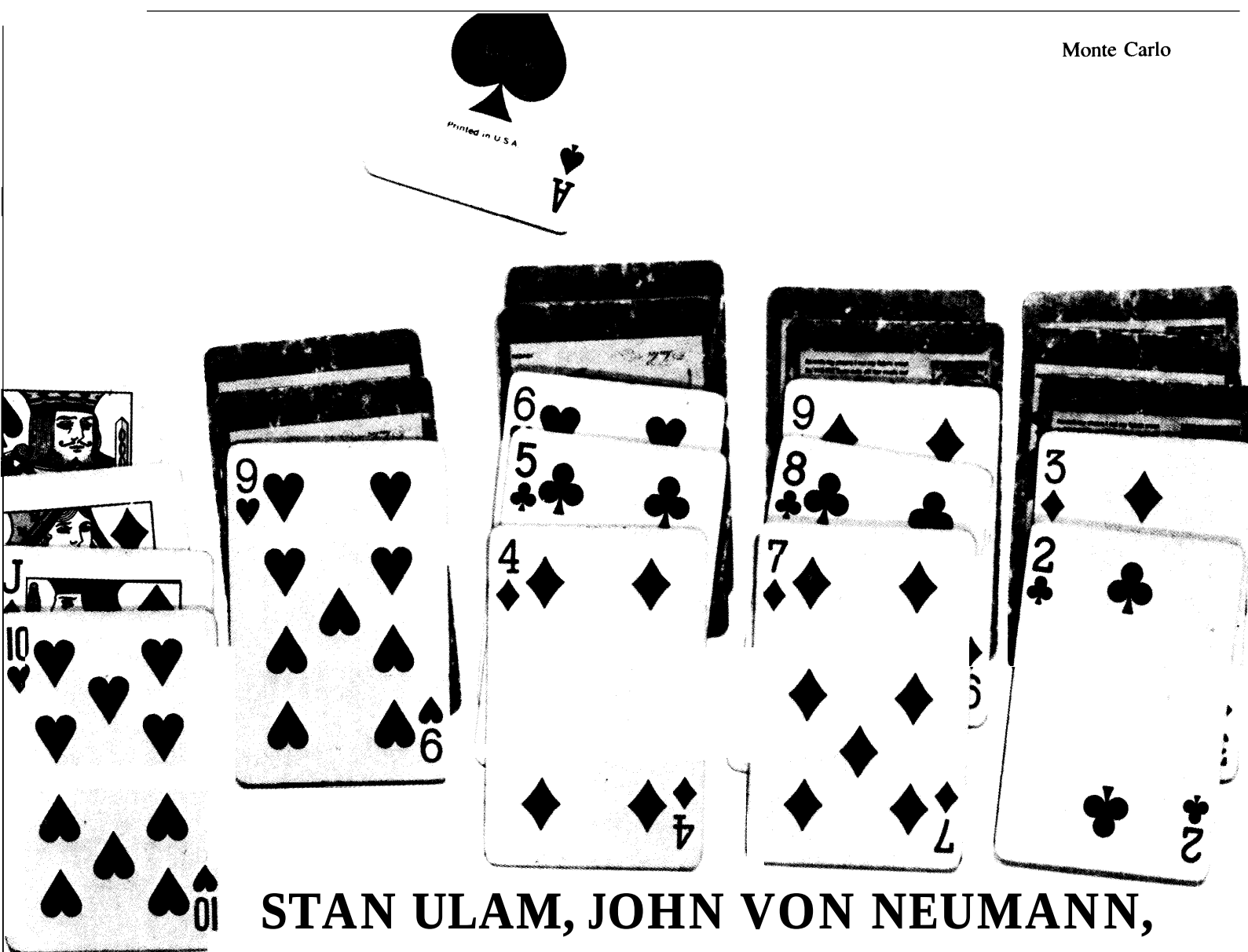




STAN ULAM, JOHN VON NEUMANN, and the MONTE CARLO METHOD

by Roger Eckhardt

The Monte Carlo method is a statistical sampling technique that over the years has been applied successfully to a vast number of scientific problems. Although the computer codes that implement Monte Carlo have grown ever more sophisticated, the essence of the method is captured in some unpublished remarks Stan made in 1983 about solitaire.

"The first thoughts and attempts I made to practice [the Monte Carlo method] were suggested by a question which occurred to me in 1946 as I was convalescing from an illness and playing solitaires. The question was what are the chances that a Canfield solitaire laid out with 52 cards will come out successfully? After spending a lot of time trying to estimate them by pure

combinatorial calculations, I wondered whether a more practical method than "abstract thinking" might not be to lay it out say one hundred times and simply observe and count the number of successful plays. This was already possible to envisage with the beginning of the new era of fast computers, and I immediately thought of problems of neutron diffusion and other questions of mathematical physics, and more generally how to change processes described by certain differential equations into an equivalent form interpretable as a succession of random operations. Later... [in 1946, I] described the idea to John von Neumann and we began to plan actual calculations."

Von Neumann was intrigued. Statistical sampling was already well known

in mathematics, but he was taken by the idea of doing such sampling using the newly developed electronic computing techniques. The approach seemed especially suitable for exploring the behavior of neutron chain reactions in fission devices. In particular, neutron multiplication rates could be estimated and used to predict the explosive behavior of the various fission weapons then being designed.

In March of 1947, he wrote to Robert Richtmyer, at that time the Theoretical Division Leader at Los Alamos (Fig. 1). He had concluded that "the statistical approach is very well suited to a digital treatment," and he outlined in some detail how this method could be used to solve neutron diffusion and multiplication problems in fission devices for the case "of 'inert' criticality" (that is, approximated as momentarily static config-

March 11, 1947

VIA AIRMAIL: REGISTERED

Mr. R. Richtmyer
 Post Office Box 1885
 Santa Fe, New Mexico

Dear Bob:

This is the letter I promised you in the course of our telephone conversation on Friday, March 7th.

I have been thinking a good deal about the possibility of using statistical methods to solve neutron diffusion and multiplication problems, in accordance with the principle suggested by Stan Ulam. The more I think about this, the more I become convinced that the idea has great merit. My present conclusions and expectations can be summarized as follows:

- (1) The statistical approach is very well suited to a digital treatment. I worked out the details of a criticality discussion under the following conditions:

- (a) Spherically symmetric geometry
- (b) Variable (if desired, continuously variable) position along the radial (28 or Be or radius, of active material)

Page Eleven

ables can be inserted in a subsequent operation, and the cards with $r = 0$ corresponding neutrons that were absorbed within the assembly) as well as those with $i = N + 1$ (i.e., corresponding to neutrons that escaped from the assembly), may be sorted out.

The manner in which this material can then be used for all kinds of neutron statistic investigations is obvious.

I append a tentative "computing sheet" for the calculation above. It is, of course, neither an actual "computing sheet" for a (human) computer group, nor a set-up for the ENIAC, but I think that it is well suited to serve as a basis for either. It should give a reasonably immediate idea of the amount of work that is involved in the procedure in question.

I cannot assert this with certainty yet, but it seems to me very likely that the instructions given on this "computing sheet" do not exceed the 'logical' capacity of the ENIAC. I doubt that the processing of 100 'neutrons' will take much longer than the reading, punching and (once) sorting time of 100 cards; i.e., about 5 minutes. Hence, taking 100 'neutrons' through 100 of these stages should take about 300 minutes; i.e., 5 hours.

Please let me know what you and Stan think of these things. Does the approach and the formulation and generality of the criticality problem seem reasonable to you, or would you prefer some other variant?

When could this be?

With best regards;

Very truly yours,

John von Neumann

TENTATIVE COMPUTING SHEET

Date:

- (1) x_i, y_i, z_i
 as functions of $i = 1, \dots, N$ ($r_0 = 0$)
- (2) $\sum_{i=1}^N x_i, \sum_{i=1}^N y_i, \sum_{i=1}^N z_i, \sum_{i=1}^N x_i^2, \sum_{i=1}^N y_i^2, \sum_{i=1}^N z_i^2$
 as functions of $N \geq 0, \leq N_0$ (2)
- (3) r_0
- (4) $\varphi_A(r), \varphi_T(r), \varphi_S(r)$
 as functions of $r \geq 0, \leq r_0$ (2)
- (5) $-\log \lambda$
 as function of $\lambda \geq 0, \leq 1$ (2)

Card C

- 1 i
- 2 x
- 3 y
- 4 z
- 5 t

1) Tabulated. (Discrete domain.)

2) Tabulated, to be interpolated or approximated

Page 11

TABLES
 STARTED

- R₅ e^{11}
- R₆ e^{111}
- R₇ e^{1111}

Calculations:

Explanations:

Instructions:

- 1 r of $C_1 - 1$, see (1)
- 2 r of C_1 , see (1)
- 3 $(C_1)^2$
- 4 $(C_2)^2$
- 5 $\frac{3}{4}$
- 6 $C_2 \left\{ \begin{array}{l} \geq 0 \rightarrow A \\ < 0 \rightarrow B \end{array} \right.$
- 7 $(1)^2$
- 8 $5 + 1$
- 9 $\frac{5}{2} \left\{ \begin{array}{l} \geq 0 \rightarrow B' \\ < 0 \rightarrow B'' \end{array} \right.$
- 10 $\frac{1}{2} \left\{ \begin{array}{l} \geq 0 \rightarrow B' \\ < 0 \rightarrow B'' \end{array} \right.$
- 11 $\frac{1}{2} \left\{ \begin{array}{l} \geq 0 \rightarrow B' \\ < 0 \rightarrow B'' \end{array} \right.$
- 12 $(10)^2$
- 13 $5 + 12$
- 14 $11(\text{sign}) = \sqrt{13}$
- 15 $\frac{1}{4} - C_2$
- 16 x of C_1 , see (1)
- 17 y of C_1 , see (1)
- 18 z of C_1 , see (1)
- 19 $\sum_{i=1}^N x_i$ of C_4 , see (2)

r_{i-1}
 r_i
 s^2
 r^2
 $s^2 - r^2$
 $s \left\{ \begin{array}{l} \geq 0 \rightarrow A \\ < 0 \rightarrow B \end{array} \right.$
 $r_{i-1}^2 + s^2 - r^2 \leq 0$
 $r_{i-1}^2 + s^2 - r^2 > 0$
 $r_{i-1}^2 + s^2 - r^2 = 0$
 $r_{i-1}^2 + s^2 - r^2 = 1$
 $r_{i-1}^2 + s^2 - r^2 = -1$
 $r^2 + s^2 - r^2$
 s^2
 x_i
 y_i
 z_i
 $\sum_{i=1}^N x_i$

Fig. 1. The first and last pages of von Neumann's remarkable letter to Robert Richtmyer are shown above, as well as a portion of his tentative computing sheet. The last illustrates how extensively von Neumann had applied himself to the details of a neutron-diffusion calculation.

urations). This outline was the first formulation of a Monte Carlo computation for an electronic computing machine.

In his formulation von Neumann used a spherically symmetric geometry in which the various materials of interest varied only with the radius. He assumed that the neutrons were generated isotropically and had a known velocity spectrum and that the absorption, scattering, and fission cross-sections in the fissionable material and any surrounding materials (such as neutron moderators or reflectors) could be described as a function of neutron velocity. Finally, he assumed an appropriate accounting of the statistical character of the number of fission neutrons with probabilities specified for the generation of 2, 3, or 4 neutrons in each fission process.

The idea then was to trace out the history of a given neutron, using random digits to select the outcomes of the various interactions along the way. For example, von Neumann suggested that in the computation "each neutron is represented by [an 80-entry punched computer] card ... which carries its characteristics," that is, such things as the zone of material the neutron was in, its radial position, whether it was moving inward or outward, its velocity, and the time. The card also carried "the necessary random values" that were used to determine at the next step in the history such things as path length and direction, type of collision, velocity after scattering—up to seven variables in all. A "new" neutron was started (by assigning values to a new card) whenever the neutron under consideration was scattered or whenever it passed into another shell; cards were started for several neutrons if the original neutron initiated a fission. One of the main quantities of interest, of course, was the neutron multiplication rate—for each of the 100 neutrons started, how many would be present after, say, 10^{-8} second?

At the end of the letter, von Neumann attached a tentative "computing sheet" that he felt would serve as a basis for

setting up this calculation on the ENIAC. He went on to say that "it seems to me very likely that the instructions given on this 'computing sheet' do not exceed the 'logical' capacity of the ENIAC." He estimated that if a problem of the type he had just outlined required "following 100 primary neutrons through 100 collisions [each]... of the primary neutron or its descendants," then the calculations would "take about 5 hours." He further stated, somewhat optimistically, that "in changing over from one problem of this category to another one, only a few numerical constants will have to be set anew on one of the 'function table' organs of the ENIAC."

His treatment did not allow "for the displacements, and hence changes of material distribution, caused by hydrodynamics," which, of course, would have to be taken into account for an explosive device. But he stated that "I think that I know how to set up this problem, too: One has to follow, say 100 neutrons through a short time interval Δt ; get their momentum and energy transfer and generation in the ambient matter; calculate from this the displacement of matter; recalculate the history of the 100 neutrons by assuming that matter is in the middle position between its original (unperturbed) state and the above displaced (perturbed) state;... iterating in this manner until a "self-consistent" system of neutron history and displacement of matter is reached. This is the treatment of the first time interval Δt . When it is completed, it will serve as a basis for a similar treatment of the second time interval... etc., etc."

Von Neumann also discussed the treatment of the radiation that is generated during fission. "The photons, too, may have to be treated 'individually' and statistically, on the same footing as the neutrons. This is, of course, a non-trivial complication, but it can hardly consume much more time and instructions than the corresponding neutronic part. It seems

to me, therefore, that this approach will gradually lead to a completely satisfactory theory of efficiency, and ultimately permit prediction of the behavior of all possible arrangements, the simple ones as well as the sophisticated ones."

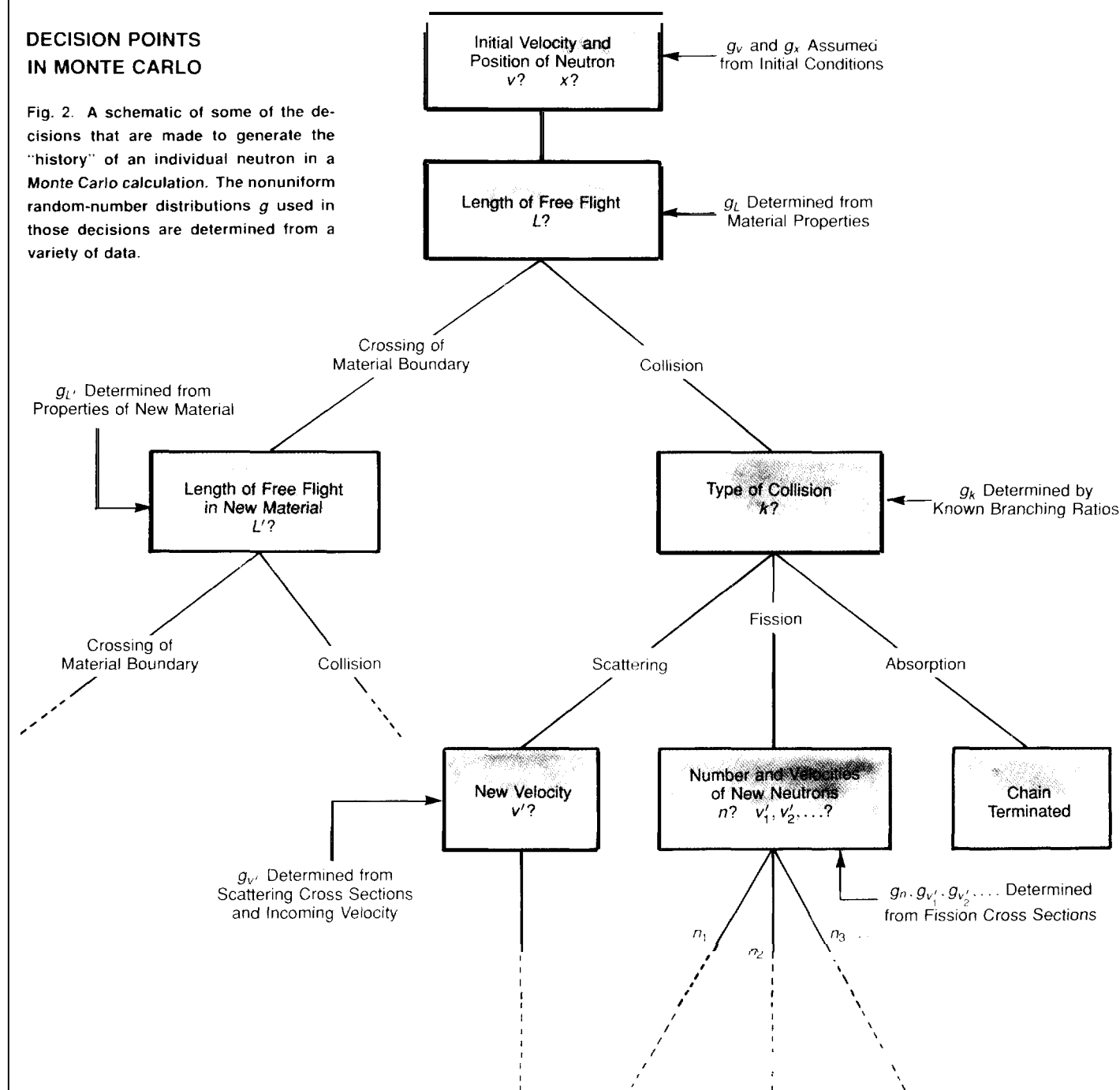
And so it has. At Los Alamos in 1947, the method was quickly brought to bear on problems pertaining to thermonuclear as well as fission devices, and, in 1948, Stan was able to report to the Atomic Energy Commission about the applicability of the method for such things as cosmic ray showers and the study of the Hamilton Jacobi partial differential equation. Essentially all the ensuing work on Monte Carlo neutron-transport codes for weapons development and other applications has been directed at implementing the details of what von Neumann outlined so presciently in his 1947 letter (see "Monte Carlo at Work").

In von Neumann's formulation of the neutron diffusion problem, each neutron history is analogous to a single game of solitaire, and the use of random numbers to make the choices along the way is analogous to the random turn of the card. Thus, to carry out a Monte Carlo calculation, one needs a source of random numbers, and many techniques have been developed that pick random numbers that are *uniformly* distributed on the unit interval (see "Random-Number Generators"). What is really needed, however, are *nonuniform* distributions that simulate probability distribution functions specific to each particular type of decision. In other words, how does one ensure that in random flights of a neutron, on the average, a fraction $e^{-x/\lambda}$ travel a distance x/λ mean free paths or farther without colliding? (For a more mathematical discussion of random variables, probability distribution functions, and Monte Carlo, see pages 68–73 of "A Tutorial on Probability, Measure, and the Laws of Large Numbers.")

The history of each neutron is gener-

DECISION POINTS IN MONTE CARLO

Fig. 2. A schematic of some of the decisions that are made to generate the "history" of an individual neutron in a Monte Carlo calculation. The nonuniform random-number distributions g used in those decisions are determined from a variety of data.



ated by making various decisions about the physical events that occur as the neutron goes along (Fig. 2). Associated with each of these decision points is a known, and usually nonuniform, distribution of random numbers g that mirrors the probabilities for the outcomes possible for the event in question. For instance, returning to the example above, the distribution of random numbers g_L used to determine the distance that a neutron trav-

els before interacting with a nucleus is exponentially decreasing, making the selection of shorter distances more probable than longer distances. Such a distribution simulates the observed exponential falloff of neutron path lengths. Similarly, the distribution of random numbers g_k used to select between a scattering, a fission, and an absorption must reflect the known probabilities for these different outcomes. The idea is to divide the

unit interval (0, 1) into three subintervals in such a way that the probability of a uniform random number being in a given subinterval equals the probability of the outcome assigned to that set.

In another 1947 letter, this time to Stan Ulam, von Neumann discussed two techniques for using uniform distributions of random numbers to generate the desired nonuniform distributions g (Fig. 3). The first technique, which had already been

ANOTHER VON NEUMANN LETTER

Fig. 3. In this 1947 letter to Stan Ulam, von Neumann discusses two methods for generating the nonuniform distributions of random numbers needed in the Monte Carlo method. The second paragraph summarizes the inverse-function approach in which (x') represents the uniform distribution and (ξ') the desired nonuniform distribution. The rest of the letter describes an alternative approach based on two uniform and independent distributions: (x') and (y') . In this latter approach a value x' from the first set is accepted when a value y' from the second set satisfies the condition $y' \leq f(x')$, where $f(\xi') d\xi$ is the density of the desired distribution function. (It should be noted that in von Neumann's example for forming the random pairs $\xi = \sin x$ and $\eta = \cos x$, he probably meant to say that x is equidistributed between 0 and 360 degrees (rather than "300"). Also, his notation for the tangent function is tg , so that the second set of equations for ξ and η are just half-angle ($y = x/2$) trigonometric identities.)

random	digits 0, ..., 9:	Replace ξ, η by	ξ, η
0			ξ, η
1	"		$-\xi, \eta$
2	"		$\xi, -\eta$
3	"		$-\xi, -\eta$
4	"		η, ξ
5	"		$\eta, -\xi$
6	"		$-\eta, \xi$
7	"		$-\eta, -\xi$
8	Reject this digit		
9	" " "		

Now $t = tg y$, $0^\circ \leq y \leq 45^\circ$, lies between 0 and 1, and its distribution function is $\frac{dt}{1+t^2}$. Hence one may pick pairs of numbers t, s both (independently) equidistributed between 0 and 1, and then use t for $(1+t^2)s \leq 1$ and s for $(1+t^2)s > 1$. reject t, s and start this step

Of course, the first part requires a divider, but the method may still be worth keeping in mind, especially when the ENIAC is available.

With best regards from house.

Yours, as ever,

John
John von Neumann

May 21, 1947

Mr. Stan Ulam
Post Office Box 1663
Santa Fe
New Mexico

Dear Stan:

Thanks for your letter of the 19th. I need not tell you that Klari and I are looking forward to the trip and visit at Los Alamos this Summer. I have already received the necessary papers from Carson Mark. I filled out and returned mine yesterday; Klari's will follow today.

I am very glad that preparations for the random numbers work are to begin soon. In this connection, I would like to mention this: Assume that you have several random number distributions, each equidistributed in $0, 1: (x'), (y'), (z'), \dots$. Assume that you want one with the distribution function (density) $f(\xi) d\xi: (\xi')$. One way to form it is to form the cumulative distribution function: $g(\xi) = \int_0^\xi f(\xi) d\xi$ to invert it $h(x) = \xi \Leftrightarrow x = g(h(x))$, and to form $\xi' = h(x')$ with this $h(x)$, or some approximant polynomial. This is, as I see, the method that you have in mind.

An alternative, which works if $f(\xi)$ and all values of $f(\xi)$ lie in $0, 1$, is this: Scan pairs x', y' and use or reject x', y' according to whether $y' \leq f(x')$ or not. In the first case, put $\xi' = x'$ in the second case form no ξ' at that step.

The second method may occasionally be better than the first one. In some cases combinations of both may be best; e.g., form random pairs $\xi = \sin x, \eta = \cos x$ with x equidistributed between 0° and 300° . The obvious way consists of using the sin - cos - tables (with interpolation). This is clearly closely related to the first method. This is an alternative procedure:

Put $\xi = \frac{2t}{1+t^2}, \eta = \frac{1-t^2}{1+t^2}, t = tg y$, with y (which is $\frac{x}{2}$) equidistributed between 0° and 180° . Restrict y to 0° to 45° . Then the ξ, η will have to be replaced randomly by η, ξ and again by $\pm \xi, \pm \eta$. This can be done by using random digits 0, ..., 7. It is also feasible with

proposed by Stan, uses the inverse of the desired function $f = g^{-1}$. For example, to get the exponentially decreasing distribution of random numbers on the interval $(0, \infty)$ needed for path lengths, one applies the inverse function $f(x) = -\ln x$ to a uniform distribution of random numbers on the open interval $(0, 1)$.

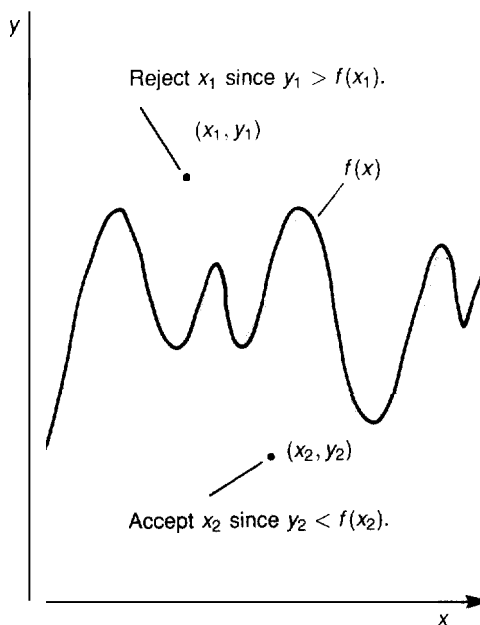
What if it is difficult or computationally expensive to form the inverse function, which is frequently true when the desired function is empirical? The rest of von Neumann's letter describes an alternative technique that will work for such cases. In this approach *two* uniform and independent distributions (x^i) and (y^i) are used. A value x^i from the first set is accepted when a value y^i from the second set satisfies the condition $y^i \leq f(x^i)$, where $f(x^i)dx^i$ is the density of the desired distribution function (that is, $g(x) = \int f(x)dx$).

This acceptance-rejection technique of von Neumann's can best be illustrated graphically (Fig. 4). If the two numbers x^i and y^i are selected randomly from the domain and range, respectively, of the function f , then each pair of numbers represents a point in the function's coordinate plane (x^i, y^i) . When $y^i > f(x^i)$ the point lies above the curve for $f(x)$, and x^i is rejected; when $y^i \leq f(x^i)$ the point lies on or below the curve, and x^i is accepted. Thus, the fraction of accepted points is equal to the fraction of the area below the curve. In fact, the proportion of points selected that fall in a small interval along the x -axis will be proportional to the average height of the curve in that interval, ensuring generation of random numbers that mirror the desired distribution.

After a series of "games" have been played, how does one extract meaningful information? For each of thousands of neutrons, the variables describing the chain of events are stored, and this collection constitutes a numerical model of the process being studied. The collection of variables is analyzed using sta-

THE ACCEPTANCE-REJECTION METHOD

Fig. 4. If two independent sets of random numbers are used, one of which (x^i) extends uniformly over the range of the distribution function f and the other (y^i) extends over the domain of f , then an acceptance-rejection technique based on whether or not $y^i \leq f(x^i)$ will generate a distribution for (x^i) whose density is $f(x^i)dx^i$.



tistical methods identical to those used to analyze experimental observations of physical processes. One can thus extract information about any variable that was accounted for in the process. For example, the average energy of the neutrons at a particular time is calculated by simply taking the average of all the values generated by the chains at that time. This value has an uncertainty proportional to $\sqrt{V/(N-1)}$, where V is the variance of, in this case, the energy and N is the number of trials, or chains, followed.

It is, of course, desirable to reduce statistical uncertainty. Any modification to the stochastic calculational process that generates the same expectation values but smaller variances is called a variance-

reduction technique. Such techniques frequently reflect the addition of known physics to the problem, and they reduce the variance by effectively increasing the number of data points pertinent to the variable of interest.

An example is dealing with neutron absorption by weighted sampling. In this technique, each neutron is assigned a unit "weight" at the start of its path. The weight is then decreased, bit by bit at each collision, in proportion to the absorption cross section divided by the total collision cross section. After each collision an outcome *other* than absorption is selected by random sampling and the path is continued. This technique reduces the variance by replacing the sudden, one-time process of neutron absorption by a gradual elimination of the neutron.

Another example of variance reduction is a technique that deals with outcomes that terminate a chain. Say that at each collision *one* of the alternative outcomes terminates the chain and associated with this outcome is a particular value x_i for the variable of interest (an example is x_i being a path length long enough for the neutron to escape). Instead of collecting these values *only* when the chain terminates, one can generate considerably more data about this particular outcome by making an extra calculation at *each* decision point. In this calculation the known value x_i for termination is multiplied by the probability that that outcome will occur. Then *random* values are selected to continue the chain in the usual manner. By the end of the calculation, the "weighted values" for the terminating outcome have been summed over all decision points. This variance-reduction technique is especially useful if the probability of the alternative in question is low. For example, shielding calculations typically predict that only one in many thousands of neutrons actually get through the shielding. Instead of accumulating those rare paths, the small probabilities that a neutron will get through the shield on its

very next free flight are accumulated after each collision.

The Monte Carlo method has proven to be a powerful and useful tool. In fact, “solitaire games” now range from the neutron- and photon-transport codes through the evaluation of multi-dimensional integrals, the exploration of the properties of high-temperature plasmas, and into the quantum mechanics of systems too complex for other methods.

Quite a handful. ■



Random-Number Generators

by Tony Warnock

Random numbers have applications in many areas: simulation, game-playing, cryptography, statistical sampling, evaluation of multiple integrals, particle-transport calculations, and computations in statistical physics, to name a few. Since each application involves slightly different criteria for judging the “worthiness” of the random numbers generated, a variety of generators have been developed, each with its own set of advantages and disadvantages.

Depending on the application, three types of number sequences might prove adequate as the “random numbers.” From a purist point of view, of course, a series of numbers generated by a truly random process is most desirable. This type of sequence is called a *random-number sequence*, and one of the key problems is deciding whether or not the generating process is, in fact, random. A more practical sequence is the *pseudo-random sequence*, a series of numbers generated by a deterministic process that is intended merely to imitate a random sequence but which, of course, does not rigorously obey such things as the laws of large numbers (see page 69). Finally, a *quasi-random sequence* is a series of numbers that makes no pretense at being random but that has important predefined statistical properties shared with random sequences.

Physical Random-Number Generators

Games of chance are the classic examples of random processes, and the first inclination would be to use traditional gambling devices as random-number generators. Unfortunately, these devices are rather slow, especially since the typical computer application may require millions of numbers per second. Also, the numbers obtained from such devices are not always truly random: cards may be imperfectly shuffled, dice may not be true, wheels may not be balanced, and so forth. However, in the early 1950s the Rand Corporation constructed a million-digit table of random numbers using an electrical “roulette wheel.” (The device had 32 slots, of which 12 were ignored; the others were numbered from 0 to 9 twice.)

Classical gambling devices appear random only because of our ignorance of initial conditions; in principle, these devices follow deterministic Newtonian physics. Another possibility for generating truly random numbers is to take advantage of the Heisenberg uncertainty principle and quantum effects, say by counting decays of a radioactive source or by tapping into electrical noise. Both of these methods have been used to generate random numbers for computers, but both suffer the defects of slowness and ill-defined distributions (however, on a different but better order of magnitude than gambling devices).

For instance, although each decay in a radioactive source may occur randomly and independently of other decays, it is not necessarily true that successive counts in the detector are independent of each other. The time it takes to reset the counter, for example, might depend on the previous count. Furthermore, the source itself constantly changes in time as the number of remaining radioactive particles decreases exponentially. Also, voltage drifts can introduce bias into the noise of electrical devices.

There are, of course, various tricks to overcome some of these disadvantages. One can partially compensate for the counter-reset problem by replacing the string of bits that represents a given count with a new number in which all of the original 1-1 and 0-0 pairs have been discarded and all of the original 0-1 and 1-0 pairs have been changed to 0 and 1, respectively. This trick reduces the bias caused when the probability of a 0 is different from that of a 1 but does not completely eliminate nonindependence of successive counts.

A shortcoming of *any* physical generator is the lack of reproducibility. Reproducibility is needed for debugging codes that use the random numbers and for making correlated or anti-correlated computations. Of course, if one wants random numbers for a cryptographic one-time pad, reproducibility is the last attribute desired, and time can be traded for security. A radioactive source used with the bias-removal technique described above is probably sufficient.

Arithmetical Pseudo-Random Generators

The most common method of generating pseudo-random numbers on the computer uses a recursive technique called the linear-congruential, or Lehmer, generator. The sequence is defined on the set of integers by the recursion formula

$$x_{n+1} = Ax_n + C \pmod{M},$$

where x_n is the n th member of the sequence, and A , C , and M are parameters that can be adjusted for convenience and to ensure the pseudo-random nature of the sequence. For example, M , the modulus, is frequently taken to be the word size on the computer, and A , the multiplier, is chosen to yield both a long period for the sequence and good statistical properties.

When M is a power of 2, it has been shown that a suitable sequence can be generated if, among other things, C is odd and A satisfies $A \equiv 5 \pmod{8}$ (that is, $A - 5$ is a multiple of 8). A simple example of the generation of a 5-bit number sequence using these conditions would be to set $M = 32$ (5 bits), $A = 21$, $C = 1$, and $x_0 = 13$. This yields the sequence

$$13, 18, 27, 24, 25, 14, 7, 20, 5, 10, \dots,$$

or, in binary,

$$01101, 10010, 11011, 11000, 11001, 01110, 00111, 10100, 00101, 01010, \dots \quad (1)$$

This type of generator has the interesting (or useful, or disastrous) property, illustrated by Seq. 1, that the least significant bit always has the alternating pattern 101010.... Further, the next bit has a pattern with period 4 (0110 above), the third bit has period 8, and so forth. Ultimately, the most significant bit has period M , which becomes the period of the sequence itself. Our example uses a short 5-bit word, which generates a sequence with a period of only 32. It is not unusual in many computer applications, however, to use many more bits (for example, to use a 32-bit word to generate a sequence with period $M = 2^{32}$).

One must be careful not to use such sequences in a problem with structures having powers of 2 in their dimensions. For example, a sequence with period 2^{32} would be a poor choice if the problem involved, say, a 3-dimensional lattice with sides of 128 ($= 2^7$) because the structure of the sequence can then interact unfavorably with the structure of the problem. Furthermore, there would be only $2^{32}/(2^7)^3 = 2048$ possible states. The usual assumption in Monte Carlo computations is that one has used a “representative” sample of the total number of possible computations—a condition that is certainly not true for this example.

One method of improving a pseudo-random-number generator is to combine two or more unrelated generators. The length of the hybrid will be the least common multiple of the lengths of the constituent sequences. For example, we can use the theory of *normal numbers* to construct a sequence that has all the statistical features of a “truly random” sequence and then combine it with a linear-congruential sequence. This technique yields a hybrid possessing the strengths of both sequences—for example, one that retains the statistical features of the normal-number sequence.

We first construct a normal number, that is, a number in base b for which each block of K digits has limiting frequency $(1/b)^K$. A simple example in base 2 can be constructed by concatenating the sequence of integers

1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111, ...

to form the normal number

1101110010111011110001001 101010111100110111101111

If the number is blocked into 5-digit sets

11011,10010, 11101, 11100,01001, 10101,01111,00110,11 110, 11111,..., (2)

it becomes a sequence of numbers in base 2 that satisfy all linear statistical conditions for randomness. For example, the frequency of a specific 5-bit number is $(1/2)^5$.

Sequences of this type do not “appear” random when examined; it is easy for a person to guess the rule of formation. However, we can further disguise the sequence by combining it with the linear-congruence sequence generated earlier (Seq. 1). We do this by performing an *exclusive-or* (XOR) operation on the two sequences:

01101, 10010, 11011, 11000, 11001, 01110, 00111, 10100, 00101, 01010, ... (1)

and

11011, 10010, 11101, 11100, 01001, 10101, 01111, 00110, 11110, 111 11, ... (2)

yield

10110, 00000, 00110, 00100, 10000, 11011, 01000, 10010, 11011, 10101, (3)

Of course, if Seq. 3 is carried out to many places, a pattern in it will also become apparent. To eliminate the new pattern, the sequence can be XOR'ed with a third pseudo-random sequence of another type, and so on.

This type of hybrid sequence is easy to generate on a binary computer. Although for most computations one does not have to go to such pains, the technique is especially attractive for constructing "canonical" generators of apparently random numbers.

A key idea here is to take the notion of randomness to mean simply that the sequence can pass a given set of statistical tests. In a sequence based on normal numbers, each term will depend nonlinearly on the previous terms. As a result, there are nonlinear statistical tests that can show the sequence not to be random. In particular, a test based on the transformations used to construct the sequence itself will fail. But, the sequence will pass all *linear* statistical tests, and, on that level, it can be considered to be random.

What types of linear statistical tests are applied to pseudo-random numbers? Traditionally, sequences are tested for uniformity of distribution of single elements, pairs, triples, and so forth. Other tests may be performed depending on the type of problem for which the sequence will be used. For example, just as the correlation between two sequences can be tested, the auto-correlation of a single sequence can be tested after displacing the original sequence by various amounts. Or the number of different types of "runs" can be checked against the known statistics for runs. An increasing run, for example, consists of a sequential string of increasing numbers from the generator (such as, 0.08, 0.21, 0.55, 0.58, 0.73, . . .). The waiting times for various events (such as the generation of a number in each of the five intervals (0, 0.2), (0.2, 0.4), ..., (0.8, 1)) may be tallied and, again, checked against the known statistics for random-number sequences.

If a generator of pseudo-random numbers passes these tests, it is deemed to be a "good" generator, otherwise it is "bad." Calling these criteria "tests of randomness" is misleading because one is testing a hypothesis known to be false. The usefulness of the tests lies in their similarity to the problems that need to be solved using the stream of pseudo-random numbers. If the generator fails one of the simple tests, it will surely not perform reliably for the real problem. (Passing all such tests may not, however, be enough to make a generator work for a given problem, but it makes the programmers setting up the generator feel better.)

Quasi-Random Numbers

For some applications, such as evaluating integrals numerically, the use of quasi-random sequences is much more efficient than the use of either random or pseudo-random sequences. Although quasi-random sequences do not necessarily mimic a random sequence, they can be tailored to satisfy the equi-distribution criteria needed for the integration. By this I mean, roughly speaking, that the numbers are spread throughout the region of interest in a much more uniform manner than a random or pseudo-random sequence.

For example, say one needs to find the average of the quantity $f(x)$ over the set of coordinates x , knowing the distribution of coordinate values $\rho(x)$ for the system being considered. Ordinarily, the average is given by the expression

$$\langle f \rangle = \frac{\int \rho(x) f(x) dx}{\int \rho(x) dx}.$$

Rather than evaluating this integral, however, one can evaluate $f(x)$ at a series of random points. If the probability of picking a particular point x is proportional to the statistical weight $\rho(x)$, then $\langle f \rangle$ is given by the expression

$$\langle f \rangle = \sum_{i=1}^N f(x_i) / N,$$

where N is the total number of points chosen. This idea is the basis of the Metropolis technique of evaluating integrals by the Monte Carlo method,

Now if the points are taken from a random or a pseudo-random sequence, the statistical uncertainty will be proportional to $1/\sqrt{N}$. However, if a quasi-random sequence is used, the points will occupy the coordinate space with the correct distribution but in a more uniform manner, and the statistical uncertainty will be proportional to $1/N$. In other words, the uncertainty will decrease much faster with a quasi-random sequence than with a random or pseudo-random sequence.

How are quasi-random sequences generated? One type of sequence with a very uniform distribution is based on the radical-inverse function. The radical-inverse function $\phi(N, b)$ of a number N with base b is constructed by

1. writing the number in base b (for example, 14 in base 3 is 112);
2. reversing the digits (112 becomes 211); and
3. writing the result as a fraction less than 1 in base b (211 becomes 211/1000 in base 3 and, thus, $\phi(14, 3) = .211$).

A sequence based on the radical-inverse function is generated by choosing a prime number as the base b and finding $\phi(1, b), \phi(2, b), \phi(3, b), \phi(4, b), \dots$. For a problem with k dimensions, the first k primes are used, and $(\phi(N, b_1), \phi(N, b_2), \dots, \phi(N, b_k))$ becomes the N th point of the k -dimensional sequence. This sequence has a very uniform distribution and is useful in multiple integration or multi-dimensional sampling.

There are many other types of random, pseudo-random, or quasi-random sequences than the ones I have discussed here, and there is much research aimed at generating sequences with the properties appropriate to the desired application. However, the examples I have discussed should illustrate both the approaches being taken and the obstacles that must be overcome in the quest of suitable "random" numbers. ■

Monte Carlo at Work

by *Gary D. Doolen*
and *John Hendricks*

Every second nearly 10,000,000,000 “random” numbers are being generated on computers around the world for Monte Carlo solutions to problems that Stan Ulam first dreamed of solving forty years ago. A major industry now exists that has spawned hundreds of full-time careers invested in the fine art of generating Monte Carlo solutions—a livelihood that often consists of extracting an answer out of a noisy background. Here we focus on two of the extensively used Monte Carlo solvers: MCNP, an internationally used neutron and photon transport code maintained at Los Alamos; and the “Metropolis” method, a popular and efficient procedure for computing equilibrium properties of solids, liquids, gases, and plasmas.

MCNP

In the fifties, shortly after the work on the Monte Carlo method by Ulam, von Neumann, Fermi, Metropolis, Richtmyer, and others, a series of Monte Carlo transport codes began emerging from Los Alamos. The concepts on which these codes were based were those outlined by von Neumann (see “Stan Ulam, John von Neumann, and the Monte Carlo Method”), but a great deal of detailed work was needed to incorporate the appropriate physics and to develop shorter routes to statistically valid solutions.

From the beginning the neutron transport codes used a general treatment of the geometry, but successive versions added such features as cross-section libraries, variance-reduction techniques (essentially clever ways to bias the random numbers so that the guesses will cluster around the correct solution), and a free-gas model treating thermalization of the energetic fission neutrons. Also, various photon transport codes were developed that dealt with photon energies from as low as 1 kilo-electron-volt to the high energies of gamma rays. Then, in 1973, the neutron transport and the photon transport codes were merged into one. In 1977 the first version of MCNP appeared in which photon cross sections were added to account for production of gamma rays by neutron interactions. Since then the code has been distributed to over two hundred institutions worldwide.*

The Monte Carlo techniques and data now in the MCNP code represent over three hundred person-years of effort and have been used to calculate many tens of thousands of practical problems by scientists throughout the world. The types of problems include the design of nuclear reactors and nuclear safeguard systems, criticality analyses, oil well logging, health-physics problems, determinations of radiological doses, spacecraft radiation modeling, and radiation damage studies. Research on magnetic fusion has used MCNP heavily.

The MCNP code features a general three-dimensional geometry, continuous energy or multigroup physics packages, and sophisticated variance reduction techniques. Even very complex geometry and particle transport can be modeled almost exactly. In fact, the complexity of the geometry that can be represented is limited only by the dedication of the user.

*The MCNP code and manual can be obtained from the Radiation Shielding Information Center (RSIC), P.O. Box X, Oak Ridge, TN 37831.



The Metropolis Method

The problem of finding the energy and configuration of the lowest energy state of a system of many particles is conceptually simple. One calculates the energy of the system, randomly moves each particle a small distance, and recalculates the energy. If the energy has decreased, the new position is accepted, and the procedure continues until the energy no longer changes.

The question of how to calculate equilibrium properties of a finite system at a given temperature is more difficult, but it was answered in a 1953 *Journal of Chemical Physics* article by Metropolis, Rosenbluth, Rosenbluth, Teller, and Teller, who decided that the calculation should follow the same steps for finding the minimum energy but with one important change. When a move results in an *increased* energy, one accepts the new position with probability $e^{-\Delta E/T}$, where ΔE is the change in energy and T is the temperature. This procedure gives the equilibrium solution for any physical system. In fact, a system with many particles can be solved with only a few lines of code and a fast computer.

Although calculations for short-range forces are much easier than for long-range forces (such as the Coulomb force), the Metropolis technique has been used for most physical systems in which the forces between particles are known. Wayne Slattery, Hugh DeWitt, and one of the authors (GD) applied the technique to a neutral Coulomb plasma consisting of thousands of particles in a periodic box. The purpose was to calculate such physical properties as the temperature at which this type of plasma freezes and the pair distribution function, which is the probability of finding one particle at a given distance from another (see accompanying figure). Because the uncertainty in a Monte Carlo result is proportional to $1/\sqrt{N}$, where N is the number of moves of a single particle, several million moves requiring several hundred Cray hours were needed to obtain accurate results for the plasma at many temperatures.

As computers become faster and their memories increase, larger and more complicated systems are being calculated far more accurately than even Stan Ulam probably expected. ■

PAIR-DISTRIBUTION FUNCTION

This plot gives the probability of pairs of charged particles in a plasma being separated by a certain distance. The probabilities are plotted as a function of the distance between the pair of particles (increasing from left to right) and temperature (decreasing from front to back). At the left edge, both the distance and the probability are zero; at the right edge, the probability has become constant in value. Red indicates probabilities below this constant value, yellow and green above. As the temperature of the plasma decreases, lattice-like peaks begin to form in the pair-distribution function. The probabilities, generated with the Metropolis method described in the text, have been used for precise tests of many theoretical approximations for plasma models.